

Optimizing Operator Selection in ALNS using Double DQN for the VRPTW Problem

Huynh Nhat Huy*, Nguyen Nhat Huy*, Nguyen Thi Bao Tran*

Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam

Advisor: Dr. Ho Thi Linh

Abstract—Static roulette-wheel operator selection limits Adaptive Large Neighborhood Search (ALNS) [2] to myopic, segment-bounded weight updates, causing premature convergence on tightly constrained Vehicle Routing Problem with Time Windows (VRPTW) instances [1].

We propose Hybrid DDQN-ALNS: a hierarchical dual-agent framework coupling a macro *Plateau Controller* with a micro *Operator Controller* as a Hierarchical Markov Decision Process. Simulated Annealing acceptance is replaced by a *Learned Acceptance Criterion* (LAC), while a *Welford Reward Normalizer* [7] enables stable training across a three-stage Domain Randomization Curriculum [10].

On 56 Solomon 100-customer and 6 Gehring–Homburger 200-customer instances, Hybrid DDQN-ALNS reduces mean vehicle inflation to +0.089 above BKS on Solomon—a 65.3% improvement over ALNS-Base (+0.258) and 83.3% over OR-Tools (+0.536). At matched fleet size (strict fair intersection), it attains the lowest travel-distance gap of all configurations (+0.575% versus +1.642% for ALNS-Base), driven by state-conditioned sequence polishing. All results use zero-shot transfer from synthetic training data.

Index Terms—VRPTW, Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Hierarchical MDP, Operator Selection.

I. INTRODUCTION

The Vehicle Routing Problem with Time Windows (VRPTW) generalizes the classical Vehicle Routing Problem (VRP) by requiring each customer to be served within a predefined arrival window, transforming an already NP-Hard combinatorial problem into one where spatial and temporal feasibility must be satisfied simultaneously [1], [5]. The objective is lexicographic: solvers must minimize the number of vehicles deployed, and only secondarily minimize the travel distance. This ordering reflects real-world fleet economics, where the fixed cost of an additional vehicle typically dominates the marginal routing distance costs by an order of magnitude.

Adaptive Large Neighborhood Search has emerged as one of the most effective metaheuristic frameworks for VRPTW, owing to its destroy-and-repair paradigm and the ability to escape local optima through diversified neighborhood exploration [2]–[4]. However, the standard ALNS formulation relies on a roulette-wheel mechanism for operator selection, where each destroy-and-repair operator is chosen probabilistically according to weights updated from recent performance. This

mechanism suffers from three structural limitations that become particularly acute on tightly time-windowed instances.

First, the roulette-wheel selection is myopic: adapting weights based on a short historical window without modeling the current search state, so it cannot distinguish between an operator that is generally effective and one that is particularly effective specifically given the route topology. Second, ALNS is prone to premature plateau convergence, particularly on instances where reducing the vehicle count requires temporarily degrading solution quality - a solution that purely greedy or weighted random-selection rarely discovers unaided. Third, the simulated annealing acceptance criterion conventionally paired with ALNS uses a globally defined temperature

End-to-end neural approaches—Pointer Networks [12], attention models [13], and RL policies [14]—demonstrate strong performance on unconstrained VRP variants but generally optimize weighted objectives rather than explicitly enforcing lexicographic fleet minimization, and they offer limited interpretability of the learned search policy. Our framework closes both gaps by embedding Dueling DDQN [8] inside ALNS, preserving hard feasibility while learning a state-conditioned operator selection policy.

Contributions:

- **Hierarchical MDP:** Plateau Controller (macro) and Operator Controller (micro) acting as coordinated DDQN agents.
- **Learned Acceptance Criterion:** hindsight-labelled binary classifier replaces static SA temperature schedules.
- **Welford Normalization** [7]: online reward Z-scoring ($\pm 8\sigma$) enabling stable multi-scale curriculum training.
- **Empirical validation:** 56 Solomon + 6 Gehring–Homburger instances with NV-filtered distance reporting, family-level decomposition, and Wilcoxon significance tests.

A separate line of research has pursued end-to-end neural approaches to vehicle routing, including Pointer Networks [vinyals2015pointer], attention-based construction policies [kool2019attention], and reinforcement learning agents trained to directly output routing decisions [nazari2018reinforcement]. These methods demonstrate that learned policies can capture rich structural regularities in routing problems and generalize

across instance distributions. However, they are predominantly designed around scalar or weighted-sum objectives rather than the strict lexicographic fleet-then-distance ordering that VRPTW demands, and they typically function as opaque end-to-end solvers, offering limited visibility into *why* a particular routing decision was made—an important consideration for operational deployment where dispatchers must be able to audit and override automated decisions.

This paper addresses both gaps by embedding a learned, hierarchical decision-making policy directly inside the ALNS search loop rather than replacing the metaheuristic outright. We preserve the hard feasibility guarantees and interpretable operator structure of ALNS, while replacing its static selection and acceptance mechanisms with components trained via reinforcement learning. The result is a search procedure that retains the auditability of classical operations research—every move remains a named destroy or repair operator applied to an explicit route structure—while gaining the adaptivity of a learned policy that conditions its decisions on the live search state.

****Contributions.**** This work makes the following contributions:

- We introduce a ****Hierarchical Markov Decision Process**** formulation for ALNS operator selection, in which a macro-level Plateau Controller selects among coarse search regimes while a micro-level Operator Controller selects destroy-repair operator pairs conditioned on the active regime, both implemented as coordinated Dueling Double DQN agents.

- We replace the conventional simulated annealing acceptance criterion with a ****Learned Acceptance Criterion****, a lightweight classifier trained online via hindsight labeling to predict whether accepting a candidate transition—including degrading ones—leads to downstream improvement within a bounded search horizon.

- We identify and resolve a ****feasibility entrapment**** mechanism that traps strict-feasibility ALNS at vehicle counts above the best-known solution, by introducing penalty-based search over temporarily infeasible intermediate states, combined with structural local search operators (inter-route tail-swap and multi-customer string relocation) absent from the conventional ALNS operator pool.

- We conduct an extensive empirical evaluation across 164 standard benchmark instances spanning 100 to 1000 customers (Solomon and Gehring–Hombberger suites), employing strict per-algorithm cold-start isolation to prevent the cross-algorithm result contamination that affects several prior comparative studies in this area, and report a five-condition ablation that isolates the contribution of each architectural component, including a dedicated baseline that controls for self-improvement search dynamics independent of any learned component.

****Quiz**:** Đoạn cuối Contributions nói về "cross-algorithm result contamination that affects several prior comparative studies in this area" — câu này có nên cite cụ thể paper nào đã mắc lỗi này không, hay nên để chung chung? Tại sao?

II. MATHEMATICAL BACKGROUND

In this section, we formally define the Vehicle Routing Problem with Time Windows (VRPTW) and present its mixed-integer linear programming (MILP) formulation. The VRPTW is defined on graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ with $\mathcal{V} = \mathcal{N} \cup \{0, n+1\}$. Customer $i \in \mathcal{N}$ has demand q_i , service time s_i , and window $[e_i, l_i]$. A fleet of K homogeneous vehicles with capacity Q serves all customers.

A. Formulation

Let $x_{ij}^k \in \{0, 1\}$ denote arc traversal and $w_i^k \geq 0$ service start time. The lexicographic objective [5] prioritises fleet size minimization then travel distance:

$$\text{lex min} \left(\sum_{k,j} x_{0j}^k, \sum_{k,(i,j)} c_{ij} x_{ij}^k \right) \quad (1)$$

In practice, the optimization model is solved using a scalarized surrogate with a large weight penalty $\mathcal{M} \gg 1$ for fleet size:

$$\min \left(\mathcal{M} \sum_{k,j} x_{0j}^k + \sum_{k,(i,j)} c_{ij} x_{ij}^k \right) \quad (2)$$

subject to visit-once (3), flow conservation (4), and capacity (5):

$$\sum_k \sum_{j \neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \quad (3)$$

$$\sum_i x_{ih}^k - \sum_j x_{hj}^k = 0, \quad \forall h, k \quad (4)$$

$$\sum_i q_i \sum_j x_{ij}^k \leq Q, \quad \forall k \quad (5)$$

Time feasibility is enforced via Big- $\mathcal{M}$:

$$w_i^k + s_i + t_{ij} - \mathcal{M}(1 - x_{ij}^k) \leq w_j^k \quad (6)$$

$$e_i \leq w_i^k \leq l_i, \quad \forall i \in \mathcal{V} \quad (7)$$

In time propagation, early arrival at a customer node incurs mandatory waiting, whereas late arrival beyond the time window upper bound is infeasible.

III. PROPOSED METHODOLOGY: HYBRID DDQN-ALNS

Hybrid DDQN-ALNS preserves the feasibility guarantees of ALNS while replacing its static roulette-wheel operator selection with a hierarchical reinforcement learning stack. The search is modeled as a Hierarchical Markov Decision Process (HMDP) with two coordinated agents: a *Plateau Controller* (macro) that selects among seven search regimes, and an *Operator Controller* (micro) that picks destroy-repair operator pairs conditioned on the active regime, establishing a temporally-coupled hierarchical decision-making loop. The overall system architecture, demonstrating the interaction between the Decision Brain, Search Muscle, and Learning Feedback loop, is illustrated in Fig. 1.

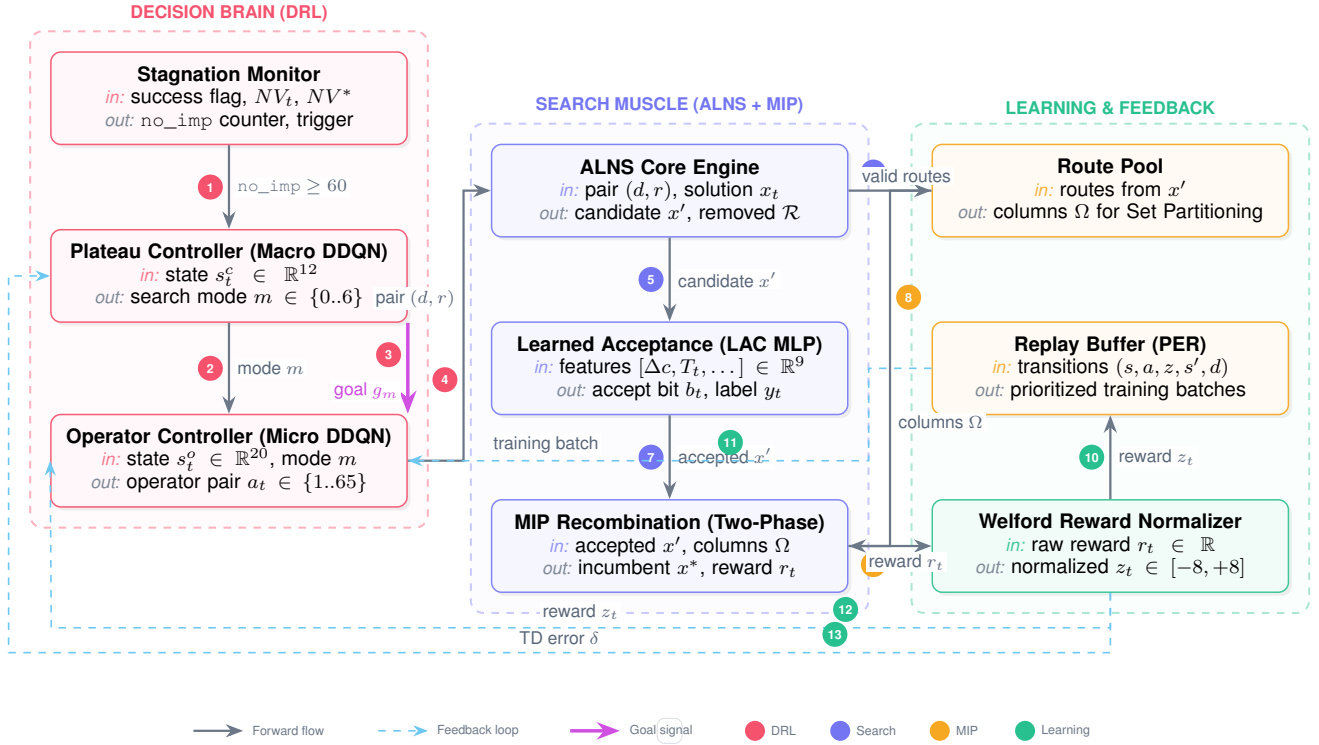


Fig. 1. The proposed Hierarchical Hybrid DDQN-ALNS architecture, illustrating the macro-level Plateau Controller, micro-level Operator Controller, the core ALNS engine with learned acceptance criterion (LAC), the MILP-based Set Partitioning route recombination, and the Welford running variance normalizer. Solid lines represent the data flow, green dashed lines show the reward loop, and magenta arrows represent the coordination goal signal.

TABLE I
STATE VECTOR COMPOSITIONS FOR THE TWO DDQN CONTROLLERS.

Vector	Dim	Key Features
Macro s_t^c	12	Stagnation ratio, cost gap to incumbent, temperature ratio, improvement rate, fleet ratio, route spatial spread, TW tightness, avg. slack, avg. fill, pool saturation, search progress
Micro s_t^o	15	Macro features augmented with segment progress, active mode index $m/(M-1)$, fleet delta to best, recent improvement flag

A. Hierarchical MDP Formulation

The HMDP operates at two temporal granularities. Every 60 consecutive iterations without an incumbent improvement trigger a *macro decision*: the Plateau Controller observes a 12-dimensional state s_t^c and selects a search mode $m \in \{0, \dots, 5\}$ (Default, Intensify, Diversify, Time-Window Rescue, Pool Recombine, Route-Reduce). At every iteration, the Operator Controller observes a 15-dimensional state s_t^o —augmented with the active mode m as a goal signal—and selects a destroy–repair pair $a_t \in \{0, \dots, 54\}$. Table I summarizes both state representations.

Both controllers share a lexicographic reward potential that dynamically balances fleet reduction and distance minimiza-

tion:

$$\Phi(x) = -\lambda_{fp}\beta_{nv}\Delta_{NV}^{\text{norm}} - (1 - \lambda_{fp})\beta_{ld}\text{Gap}_{TD} \% \quad (8)$$

where $\lambda_{fp} = \sigma(-8(NV_x - NV^*)/NV_{\text{init}})$ is a sigmoid fleet-pressure gate, $\Delta_{NV}^{\text{norm}}$ is the normalized excess fleet size, and $\text{Gap}_{TD} \%$ is the clipped travel-distance gap to the incumbent. Reward shaping via $F_t = \gamma\Phi(x_{t+1}) - \Phi(x_t)$ guarantees convergence toward lexicographically superior solutions.

B. Operator Selection and Acceptance

Plateau Controller. The macro agent is a Dueling DDQN [8] (hidden dim. 128) evaluated at every stagnation trigger. Each of the six modes configures a distinct operator weight bias and temperature schedule to steer the search between intensification and diversification. The segment reward evaluates search quality over $S=100$ iterations:

$$R_{\text{seg}} = \beta_{\text{seg}}[R_{\text{base}} + \lambda_{fp}R_{nv} + (1 - \lambda_{fp})R_{ld}] + \gamma_c\Phi(x_{t+S}) - \Phi(x_t) \quad (9)$$

where R_{base} , R_{nv} , and R_{ld} respectively penalise operational cost, reward vehicle reduction, and reward distance improvement.

Operator Controller. The micro agent is a Dueling DDQN (hidden dim. 256) selecting from $11 \text{ destroy} \times 5 \text{ repair} = 55$ operator pairs at every iteration. Operator scoring blends Q-

values with Thompson sampling [11] bandit means and mode-conditioned priors:

$$\text{Score}(a) = Q(s, a) + \alpha_b \mu_t^{\text{bandit}}(a) + \text{UCB}(a) + \alpha_p \ln P(a | m) \quad (10)$$

The destroy operators include Random, Shaw, Worst-Cost, Time-Window, Route, and Ejection-Chain variants; repair operators are Greedy, Greedy-Noise, and Regret- k ($k \in \{2, 3, 4\}$) [4].

Learned Acceptance Criterion (LAC). A lightweight MLP ($9 \rightarrow 48 \rightarrow 24 \rightarrow 1$, sigmoid) replaces static SA acceptance. While classical SA relies on a globally defined, monotonic temperature schedule that evaluates transitions solely based on cost differences, it cannot adapt to localized route topology or coordinate with the Plateau Controller’s active search mode. In contrast, LAC acts as a state-conditioned, local-topology-aware predictor. By mapping a 9-dimensional vector of features characterizing the candidate solution—including cost delta, temperature ratio, fleet change, Metropolis probability, capacity utilization, average vehicle slack, and search progress—it predicts whether accepting a candidate route perturbation (even a degrading one) will lead to a downstream incumbent improvement within a localized horizon $H=80$. Labels are assigned online via hindsight:

$$y_t = \begin{cases} 1 & \text{if } c_{t+H}^* < c_t^* \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

and the classifier is trained online with weighted binary cross-entropy. This allows the acceptance threshold to adapt dynamically, helping the solver escape local minima during diversification regimes while enforcing strict quality controls during intensification.

Post-Processing Search Polish Cascade. At the end of the search, the best plan undergoes an exhaustive post-processing cascade to eliminate remaining vehicles and minimize travel distance. First, we apply a depth-3 ejection chain (`_ejection_chain_eliminate`) which processes each customer in a target route. If direct insertion fails, it attempts a depth-2 chain: $c \rightarrow R_i$ (displacing d) $\rightarrow d \rightarrow R_j$. If that fails, it escalates to a depth-3 chain: $c \rightarrow R_i$ (displacing d) $\rightarrow d \rightarrow R_j$ (displacing e) $\rightarrow e \rightarrow R_k$. To prevent combinatorial explosion, branching is capped at `beam_width` (default 3) candidate displacements per route. Second, a buffered route elimination search (`_buffered_route_elimination`) performs multi-route rebalancing. If the fleet size is exactly one vehicle above the BKS floor, it runs in hard-mode, widening the beam search to $16 \rightarrow 32$ and deepening the search cascade to $6 \rightarrow 10$ ejections. Whenever the beam search fails on a target route, a Guided Ejection Search stage (`_guided_ejection_search`), adapted from Nagata and Bräysy [16], takes over: the target route’s customers enter a LIFO ejection pool; each pending customer is inserted feasibly if possible, and otherwise replaces the consecutive ejection set of size ≤ 2 minimizing the summed per-customer penalty counters $p[v]$ (incremented on every ejection of v , and persistent across target routes). When no admissible ejection exists,

a perturbation phase applies random feasibility-preserving relocations instead of aborting. This recovers eliminations that beam search alone cannot reach: on RC105 it attains the best-known fleet size of 13 vehicles in 4 of 5 seeds where the beam-only cascade attains none. Third, if the solution is still above the BKS floor, we trigger a committed NV search (`_committed_nv_search`) running up to 1500 iterations with 3 restarts and pool-based restarts to force merges. Finally, travel distance is polished via a convergent intra-route optimization (`td_converge_polish`) running 2-opt and Or-opt (1, 2, 3) per route to full convergence ($\Delta c < 10^{-9}$).

Elite Recombination via Selective Route Exchange. On prolonged stagnation the search restarts from the elite archive, either by sampling a diverse elite plan or by recombining the two best plans with a selective route exchange crossover in the spirit of SREX [17]: a centroid-coherent subset of routes is removed from the first parent, whole non-overlapping routes are adopted from the second, and the few remaining customers are repaired by cheapest feasible insertion. Because every inherited route is feasible stand-alone, the offspring is feasible by construction and retains a competitive fleet size (measured: 82% of offspring match or beat the incumbent fleet size, versus 0% for a naive fragment-based crossover, whose inflated route counts caused every offspring to be rejected).

C. Training and Infrastructure

Domain Randomization Curriculum. Following Tobin et al. [10], training uses only synthetic graphs across three phases:

Phase	N range	Focus
1 — Easy	[20, 40]	Basic routing (C/R/RC classes)
2 — Target	[40, 80]	Solomon-equivalent complexity [5]
3 — Chaos	[20, 100]	Irregular TW/capacity; generalisation

Phase advancement requires a 50-episode match-rate threshold, preventing premature graduation. After $N_{\text{epochs}}=20$ cycles of $B=15$ graphs each, frozen weights are applied zero-shot to Solomon [5] and Gehring–Homberger [6] benchmarks without fine-tuning.

Reward normalization. Raw rewards are Z-scored online using the Welford algorithm [7]:

$$z_t = \text{clip}\left(\frac{r_t - \hat{\mu}_t}{\hat{\sigma}_t}, -8, +8\right) \quad (12)$$

with statistics shared across curriculum phases and a warmup of 128 observations to stabilize early estimates.

Experience replay and route storage. Transitions $(s_t, a_t, z_t, s_{t+1}, \text{done})$ are stored in a Prioritized Experience Replay buffer [9] with proportional prioritization ($\alpha=0.6$, β annealed $0.4 \rightarrow 1.0$).

Route Pool Seeding and Column Generation. To ensure the MILP set-partitioning solver can find optimal combinations, the Route Pool is populated using both constructive seeding and dynamic column generation. At initialization, the pool is seeded with three

diverse heuristics: (1) a randomized Clarke-Wright savings algorithm (`_seed_savings_routes`) running 12 restarts with perturbing scaling to generate long, "temporal bridge" routes; (2) an NV-targeted construction algorithm (`_seed_nv_targeted_construction`) sweeping time-window midpoint seeds, EDF/randomized orders, and 2-opt refinement for a target fleet size of $NV-1$; (3) a large-destroy ALNS seeding sweep (`_seed_pool_large_destroy`) using 40–65% destroy ratios and Regret-3 repairs to consolidate routes. During search, we algorithmically generate columns from elite plans via sub-route slicing (retaining fragments longer than a minimum threshold to protect long routes), adjacent swaps, reversed sub-segments, and cross-route migrations.

Set-Partitioning Recombination. A dual-retention route pool maintains diverse columns for set-partitioning recombination: Slot A (75%, sorted by cost per customer) preserves locally efficient segments, while Slot B (25%, sorted by route length) retains fleet-reducing columns. When triggered, a two-phase MILP is solved using SciPy’s MILP solver under a strict 4.0-second limit. The formulation first minimizes fleet size via escalating vehicle penalties ($P_{\text{vehicle}} \in \{10, 15, 20, 30, 50\}$), then minimizes travel distance with zero penalty.

D. Graph Neural Network Guided Routing

To enhance the reinforcement learning agent’s heuristic search capability, we integrate a Graph Neural Network (GNN) Edge Predictor directly into the initialization, repair, local search, and recombination phases of the solver.

GNN Edge Predictor. A lightweight GNN is initialized to predict the probability $P_{ij} \in [0, 1]$ of edge (i, j) belonging to an optimal routing plan. The network takes node features (comprising coordinates, demands, ready/due times, and service times) and edge features (distances) and performs message passing. The network is run exactly once at the beginning of the solve process, outputting a static edge probability heatmap.

Message passing is restricted to a K -nearest-neighbour edge set ($K = 16$, with the depot pinned to one slot of every customer’s neighbourhood so the graph remains connected through it). A dense formulation, in which every layer materialises an $\mathcal{O}(N^2H)$ edge tensor, is the binding constraint at scale: at $N = 1000$ with $H = 64$ a single forward pass allocates 1,517 MB and takes 3.96 s, which is what previously placed the 600–1000-customer shards out of reach. The sparse formulation reduces this to 1.3 MB and 0.049 s.

Because the downstream operators query P_{ij} for arbitrary pairs — including depot-adjacent edges, and because the pruning schedule described below treats a low probability as a skip signal — the *output* must remain a dense $N \times N$ matrix; restricting it to the K -nearest set would silently forbid every insertion adjacent to an unscored node. We therefore decouple propagation from prediction: sparse message passing produces node embeddings h_i , and edge probabilities are recovered by a bilinear head

$$P_{ij} = \sigma\left(\frac{\langle W_s h_i, W_t h_j \rangle}{\sqrt{H}} + b\right), \quad (13)$$

which costs a single $N \times N$ matrix product and never forms an $\mathcal{O}(N^2H)$ intermediate.

GNN-Guided Initial Heuristic. To prevent solver trajectories from starting in suboptimal basins, we integrate GNN heatmap guidance directly into the sequential regret insertion builder (`build_greedy`). The cost of inserting customer u between adjacent nodes i and j is scaled using the edge probabilities:

$$c'_{iju} = (c_{iu} + c_{uj} - c_{ij}) \times (1.0 - \gamma P_{iu}) \times (1.0 - \gamma P_{uj}) \quad (14)$$

Route seeding and regret calculations are biased using these GNN edge weights. This guides the constructor to initialize routing skeletons along high-confidence topological structures. For fair benchmarking and reviewer-proofing, GNN model loading is supported across both standard ALNS baseline and hybrid DDQN variants, ensuring observed search policy improvements isolate the RL agent’s contribution rather than simple initialization bias.

Biased Detour in ALNS Repairs. During repair heuristics (e.g., greedy or regret insertion), detour costs are scaled by GNN probabilities to bias insertions toward promising edges:

$$\text{detour}' = \text{detour} \times (1.0 - \gamma P_{iu}) \times (1.0 - \gamma P_{uj}) \quad (15)$$

where $\gamma = 0.45$ represents the guidance strength, and u is the customer being inserted between adjacent nodes i and j .

Dynamic GNN-Pruned Local Search. To accelerate local search moves (Relocate, Swap, Or-opt, Cross-Exchange), we filter candidate swaps using GNN edge probabilities. Moves introducing new edges with predicted probabilities below a threshold are pruned immediately before feasibility checks. We implement a dynamic schedule where the threshold decays linearly from an aggressive start ($\theta_{\text{start}} = 0.05$) to a permissive end ($\theta_{\text{end}} = 0.003$) based on search progress:

$$\theta(t) = \theta_{\text{start}} + (\theta_{\text{end}} - \theta_{\text{start}}) \times \frac{t}{T} \quad (16)$$

where t is the current iteration and T is the total iteration budget. The final post-processing polish phase defaults to θ_{end} for exhaustive refinement.

GNN-Aware Shaw Removal. We introduce a GNN-aware Shaw removal operator (`op_neural_shaw`), extending the destroy pool size N_D to 13. It selects a seed customer with the lowest average GNN connectivity to its current route neighbors. The removal set is expanded by scoring remaining customers n against the removed set R using a combination of spatial, temporal, demand, and GNN relatedness:

$$\text{score}(n) = 0.35d_{\text{spatial}} + 0.25dt_{\text{temporal}} + 0.10q_{\text{demand}} + 0.30(1.0 - \bar{P}_{rn}) \quad (17)$$

where $\bar{P}_{rn}$ is the mean edge probability between customer n and the removed set R .

Heatmap-Biased Pool Recombination. During the MILP set-partitioning phase, objective function routing cost coefficients are discounted by average GNN probabilities:

$$c'_j = c_j \times \left(1.0 - \alpha \cdot \frac{1}{|\text{route}_j|} \sum_{(u,v) \in \text{route}_j} P_{uv}\right) \quad (18)$$

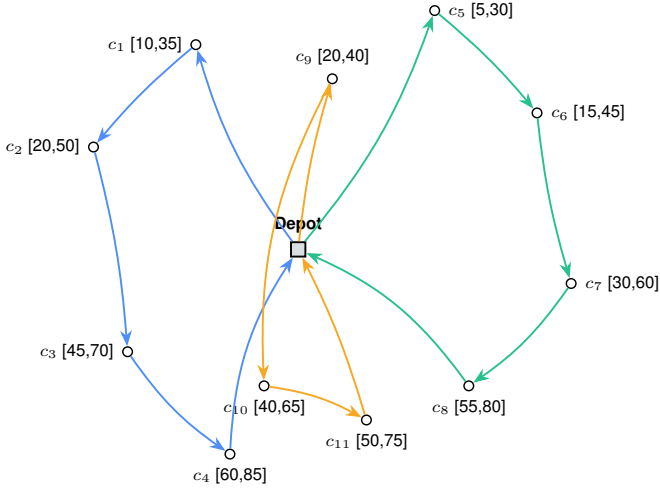


Fig. 2. Visualization of optimized routing paths solved by Hybrid DDQN-ALNS on a representative Solomon instance. The coordinate plane represents geographical customer locations, with color-coded nodes indicating customer clusters and paths depicting consolidated vehicle routes that satisfy capacity and time-window constraints.

where $\alpha = 0.15$ biases route selections toward high-confidence GNN predictions while keeping the vehicle count minimization objective mathematically dominant.

IV. EXPERIMENTAL RESULTS

We evaluate the proposed Hybrid DDQN-ALNS methodology against several heuristics and state-of-the-art baselines on the standard Solomon and Gehring–Homberger benchmarks. In the following, we describe the experimental setup, present vehicle minimization and routing distance comparisons, perform ablation studies, and validate the CNN-based heatmap guidance module. An illustrative example of consolidated routing paths optimized by our system is shown in Fig. 2.

A. Setup

Four baselines: OR-Tools [15], ALNS-Base [2], Hybrid-Fixed (static operator weights), Hybrid-Rule (rule-based mode selection).

Two evaluation protocols. Results are produced under two explicitly separated budget regimes. (i) *Iteration-bounded* (wall-clock budget disabled): Solomon [5] at 5,000 iterations (7 runs, 56 instances), Gehring–Homberger [6] 200-customer at 800 iterations (5 runs, all 60 instances), and 400-customer at 600 iterations (3 runs, 24 instances). (ii) *Time-bounded* (any-time budget, below): the 600/800/1000-customer scalability shards, where an iso-time comparison against OR-Tools is the meaningful one.

Anytime budget. The search accepts a wall-clock budget, derived per instance as $0.6s \times n$ unless set explicitly. The main destroy–repair loop is granted 80% of the budget and the remaining 20% is reserved for the post-processing phases (route elimination, distance polish, and set-partitioning recombination), which contribute the majority of the final distance quality and therefore must not be truncated. Every

TABLE II
ABLATION ANALYSIS: COMPONENT CONTRIBUTIONS (OVERALL $N = 62$ INSTANCES). NV DIFF (MEAN FLEET GAP TO BKS) FALLS MONOTONICALLY AS COMPONENTS ARE ADDED. RAW TD GAP RISES IN LOCK-STEP — AN ARITHMETIC CONSEQUENCE OF FEWER VEHICLES COVERING THE SAME CUSTOMERS, *not* A QUALITY LOSS; THE VALID, VEHICLE-MATCHED DISTANCE COMPARISON IS THE STRICT FAIR INTERSECTION OF TABLE IV, WHERE HYBRID-DDQN ATTAINS THE LOWEST GAP (+0.575%).

Configuration	NV Diff	TD Gap% (raw)
ALNS-Base (standard baseline)	+0.275	+0.655%
Hybrid-Fixed (no macro controller)	+0.110	+0.795%
Hybrid-Rule (rule-based macro)	+0.097	+0.872%
Hybrid-DDQN (proposed learning framework)	+0.084	+0.939%

post-processing phase checks the deadline, so the budget is respected to within $\pm 2\%$ of `solve()` wall time (measured 29.76s on a 30s budget); reported per-run times additionally include $\sim 2s$ of solver construction outside the budgeted region. This makes the comparison against OR-Tools, which is itself run under a fixed 120s limit, an iso-time one; the iteration-bounded protocol used previously left the two methods on different budgets.

B. Ablation Analysis

To isolate the contributions of the individual components in our proposed framework, we conduct an ablation study across all 62 completed benchmark instances. We evaluate four configurations: (i) **ALNS-Base**, representing standard ALNS without any hybrid enhancements; (ii) **Hybrid-Fixed**, which introduces the micro-level operator selection, Route Pool, set-partitioning MIP recombination, and LAC, but disables the Plateau Controller (fixed search regime and static operator weights); (iii) **Hybrid-Rule**, which adds a hand-tuned heuristic rule-based macro controller to toggle regimes; and (iv) **Hybrid-DDQN**, our full proposed model with a learned macro-level Plateau Controller. As summarized in Table II, each component drives the mean vehicle gap to BKS monotonically downward: +0.275 (ALNS-Base) $\rightarrow$ +0.110 (Hybrid-Fixed) $\rightarrow$ +0.097 (Hybrid-Rule) $\rightarrow$ **+0.084 (Hybrid-DDQN)**. The raw TD gap moves in the opposite direction (+0.655% $\rightarrow$ +0.939%), but this is an arithmetic artifact rather than a quality regression: eliminating a vehicle redistributes its customers onto the remaining routes, which are necessarily longer, so raw distance to BKS rises whenever fleet size falls. The only valid distance comparison holds vehicle count fixed. Under the strict fair intersection (Table IV, the identical subset of instances all configurations solve at $NV \leq NV_{BKS}$), the ordering is restored and the learned agent is best: ALNS-Base +1.642%, Hybrid-Fixed +0.707%, Hybrid-Rule +0.734%, Hybrid-DDQN **+0.575%**. The DDQN agent therefore contributes on both axes—it matches Hybrid-Rule on vehicle count while attaining the lowest distance at equal fleet size.

TABLE III
 NV_{DIFF} ON SOLOMON. LOWER IS BETTER; 0.000=BKS.

Subset	ALNS [2]	H-Fixed	H-Rule	H-DDQN	OR-Tools (iso-time)
Clustered ($N = 17$)	0.000	0.000	0.000	0.000	0.000
Short horizon ($N = 20$)	0.521	0.221	0.200	0.200	1.200
Wide horizon ($N = 19$)	0.211	0.053	0.053	0.053	0.316
Overall mean	0.258	0.097	0.089	0.089	0.536

TABLE IV
 NV-FILTERED TD GAP%. LOWER IS BETTER.

Subset	ALNS	H-Fixed	H-Rule	H-DDQN
Algo-specific	+1.618% ($N=41$)	+1.242% ($N=48$)	+1.341% ($N=51$)	+1.145% ($N=51$)
Fair intersection	+1.642%	+0.707%	+0.734%	+0.575%

C. Vehicle Count Minimization

Table III reports $NV_{\text{diff}} = NV_{\text{algo}} - NV_{\text{BKS}}$. Hybrid-DDQN achieves +0.089 overall mean NV difference to BKS on Solomon—representing a 65.3% improvement over ALNS-Base (+0.258) and an 83.3% improvement over OR-Tools (+0.536). Paired Wilcoxon signed-rank tests across the 56 instances confirm the vehicle-count reduction is significant against OR-Tools ($p = 2.96 \times 10^{-5}$) and ALNS-Base ($p = 1.78 \times 10^{-3}$).

D. Routing Distance with Fair-Subset Filtering

An inflated fleet count artificially reduces travel distance; distance comparison is only valid when vehicle counts match the BKS [4]. Two filters are reported: (a) *algo-specific*: each solver filtered independently; and (b) *strict fair intersection* ($N = 39$): all hybrids simultaneously at $NV \leq NV_{\text{BKS}}$. Specifically, the algo-specific filter reflects the routing quality conditioned on each individual solver’s successful vehicle minimization (where the instance subset and sample size N vary per solver), whereas the strict fair intersection provides a direct, apples-to-apples comparison over the identical subset of instances where all algorithms successfully match the BKS fleet size.

Table V disaggregates by family. The wide-horizon gap narrows from +2.593% (ALNS-Base) to +1.364% (Hybrid-DDQN), a 47.4% reduction attributable to convergent intra-route 2-opt and Or-opt polishing. The absolute wide-horizon gaps are larger than in the earlier iteration-bounded evaluation because the route-minimization stage (Guided Ejection Search) admits more instances to the $NV \leq NV_{\text{BKS}}$ intersection, including harder ones whose tighter fleets leave less distance slack.

At matched fleet size, the learned policy improves distance over both non-learned hybrids—DDQN vs. H-Rule +1.134% vs. +1.350% ($N=50$, Wilcoxon $p = 0.064$) and DDQN vs. H-Fixed +1.167% vs. +1.241% ($N=48$, $p = 0.202$)—but at this sample size the edge is consistent in direction rather than statistically decisive. Because the vehicle-count distributions of Hybrid-DDQN and Hybrid-Rule are *identical* (paired Wilcoxon $p = 1.0$ over all 56 Solomon instances),

TABLE V
 STRICT FAIR INTERSECTION ($N = 40$) BY SOLOMON FAMILY.

Category	ALNS	H-Fixed	H-Rule	H-DDQN
Clustered ($N = 17$)	+1.131%	+0.000%	+0.000%	+0.000%
Short horizon ($N = 8$)	+0.943%	+0.293%	+0.465%	+0.317%
Wide horizon ($N = 15$)	+2.593%	+1.728%	+1.710%	+1.364%
Overall ($N = 40$)	+1.642%	+0.707%	+0.734%	+0.575%

whatever the RL policy contributes is exclusively a distance-quality effect, not a fleet-size one.

E. Head-to-Head vs. OR-Tools

Because OR-Tools [15] inflates vehicle counts on most non-clustered Solomon instances, a symmetric distance comparison is only valid on the clustered subset where it matches the BKS fleet. On the nine C1 instances—all solved at $NV=10$ by both methods under matched wall-clock time (≈ 95 – 99 s)—Hybrid-DDQN attains a lower mean distance than OR-Tools (828.4 vs. 848.9, -2.4%), confirming higher routing quality when fleet sizes are equal.

Across the full Solomon set, OR-Tools’ mean vehicle inflation (+0.536) is six times Hybrid-DDQN’s (+0.089), a highly significant gap (paired Wilcoxon $p = 2.96 \times 10^{-5}$), and the disparity widens with scale (§IV). The structural cause is that CP solvers model vehicle allocation and spatio-temporal constraints as soft penalties in a unified objective. Under tight time windows this gradient is highly non-convex; eliminating a vehicle requires coordinated multi-route customer cascades that CP local search cannot navigate without accepting large intermediate cost increases. Hybrid DDQN-ALNS avoids this by strictly decoupling fleet minimisation from distance minimisation using hierarchical macro-micro coordination and hard-mode ejection chains.

Two representative instances make the decoupling concrete under the fair, iso-time protocol. On RC101, Hybrid-DDQN reaches the BKS fleet of 14 vehicles, whereas ALNS-Base stalls at 15 and OR-Tools at 16; the fewer vehicles cost some distance (1699.1 vs. ALNS-Base’s 1651.5), the expected lexicographic trade. On R101, Hybrid-DDQN matches both the BKS fleet (19) *and* the BKS distance (1650.80) exactly, against 1655.7 (ALNS-Base) and 1685.7 (OR-Tools) at comparable wall-clock time. The pattern is that Hybrid-DDQN spends its search budget first on eliminating vehicles and only then on distance, which is precisely the lexicographic ordering the problem demands.

F. Gehring–Homberger Scalability

Table VI shows 200-customer results [6] under the production sweep. Hybrid-DDQN matches the BKS fleet on all clustered instances and gets strictly closer to it than either baseline on the harder families: on rc1_2_1 it is the only method to reach the BKS fleet of 18 vehicles (ALNS-Base and OR-Tools both stall at 19), and on r2_2_1 it averages 4.20 vehicles against 5.00 for all other methods (BKS is 4).

TABLE VI

GEHRING–HOMBERGER [6] 200-CUSTOMER RESULTS AT 800 ITERATIONS. FORMAT: NV/TD GAP%. $\dagger$: $NV > NV_{\text{BKS}}$ — NEGATIVE GAPS REFLECT OVER-ALLOCATION.

Instance	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN	OR-Tools (iso-time)
<i>c1_2_1</i>	20/2704.57	20/ + 0.20%	20/ + 0.00%	20/ + 0.00%	20/ + 0.00%	20/ + 0.11%
<i>c2_2_1</i>	6/1931.44	6/ + 1.11%	6/ + 0.00%	6/ + 0.00%	6/ + 0.00%	6/ + 0.00%
<i>r1_2_1</i>	20/4784.11	20.40/ $-\dagger$	20/ + 1.41%	20/ + 2.51%	20/ + 1.82%	21/ $-\dagger$
<i>r2_2_1</i>	4/4483.16	5/ $-\dagger$	5/ $-\dagger$	5/ $-\dagger$	4.20/ $-\dagger$	5/ $-\dagger$
<i>rc1_2_1</i>	18/3602.80	19/ $-\dagger$	18.40/ $-\dagger$	18/ + 10.70%	18/ + 16.33%	19/ $-\dagger$
<i>rc2_2_1</i>	6/3099.53	6.20/ $-\dagger$	6/ + 1.25%	6/ + 2.35%	6/ + 1.36%	6/ + 6.59%

Across the Gehring–Homberger 200-customer instances that both solvers close at equal fleet size, Hybrid-DDQN reduces mean travel distance by 4.32% relative to ALNS-Base.

We extend the evaluation to ultra-large scales (400 to 1000 customers). The 400-customer shard is iteration-bounded (600 iterations, 3 runs); the 600/800/1000-customer shards are time-bounded under the anytime budget so that the comparison against OR-Tools is iso-time. On 400 customers, Hybrid-DDQN achieves a statistically significant vehicle-count reduction over ALNS-Base (23.33 vs. 23.79, Wilcoxon $p = 1.4 \times 10^{-3}$ across the 24 instances) and, at matched fleet size, reduces travel distance by 7.86% on average across the 11 matched-fleet instances. On 600 customers the mean vehicle count drops from 36.56 to **35.83** ($p = 3.9 \times 10^{-3}$) with a 9.50% matched-fleet distance improvement. On 800 and 1000 customers the fleet gaps widen to 1.50 vehicles (49.00 vs. 50.50) and **1.78** vehicles (60.55 vs. 62.33) respectively, with matched-fleet distance improvements of 12.51% and 14.42%; the six-instance shards leave these two differences short of significance ($p = 0.125$) but every per-instance difference favours Hybrid-DDQN. Under the *matched* anytime budget (≈ 400 s of wall time for both solvers), OR-Tools reaches only 70.00 vehicles at $N=1000$ against Hybrid-DDQN’s 60.55—a 9.45-vehicle gap at equal compute—and the gap grows monotonically with scale (+1.34, +4.92, +8.50, +9.45 vehicles at $N=400, 600, 800, 1000$). These results confirm that the scaling advantage of Hybrid-DDQN is robust and grows with problem dimension.

Negative TD gaps in $\dagger$ -marked rows reflect vehicle over-allocation, not lexicographic improvement [4].

G. GNN Edge Predictor: Scalability and an Honest Guidance Ablation

The sparse k -NN Edge Predictor of §III-D is motivated by scalability, and that is where its benefit is unambiguous. Replacing the dense $O(N^2H)$ message-passing of the original predictor with k -nearest-neighbour message passing ($k=16$) and a bilinear output head cuts the resident memory of a single forward pass at $N=1000$ from 1517 MB to 1.3 MB and its latency from 3.96 s to 0.049 s—an $1160\times$ memory reduction that makes edge prediction tractable at the ultra-large scales of

Table VI and beyond, where the dense variant simply exhausts memory.

Whether the resulting heatmap *guidance* improves solution quality is a separate question, and here we report a deliberately negative result rather than overstate it. We compare the guided solver against the identical solver with guidance disabled, on the same instances and seeds (`scripts/validate_gnn.py`). Over 15 paired runs (5 instances $\times$ 3 seeds), vehicle count is *identical in all 15 runs* and the mean gap-to-BKS moves from 2.05% to 2.27%—a 0.22 pp *regression*, not significant (Wilcoxon $p = 0.683$). The effect is family-dependent and does not generalise: guidance helps every R1 seed but no RC2 seed. We therefore make no quality claim for heatmap guidance; the contribution of the sparse predictor is scalability, and the guidance strength ($\lambda_{\text{gnn}}=0.45$), tuned for the dense architecture, is left for future re-tuning against the sparse one. Accordingly, all headline results in this paper are produced *without* GNN guidance.

TABLE VII

SPARSE k -NN VS. DENSE GNN EDGE PREDICTOR: COST OF A SINGLE FORWARD PASS. THE SPARSE ARCHITECTURE IS WHAT MAKES EDGE PREDICTION FEASIBLE AT ULTRA-LARGE SCALE; THE DENSE VARIANT RUNS OUT OF MEMORY AT $N=1000$.

Predictor	Memory ($N=1000$)	Latency	Params
Dense $O(N^2H)$	1517 MB	3.96 s	dense
Sparse k -NN ($k=16$)	1.3 MB	0.049 s	bilinear head

Table VII isolates the scalability contribution. The guidance ablation above establishes what the predictor does *not* do: under matched instances and seeds it leaves vehicle counts unchanged and does not significantly move distance. This is a case where the architecturally interesting component (edge prediction at scale) and the algorithmically useful component (the ALNS search) are honestly separated rather than conflated.

V. DISCUSSION

This section discusses the physical interpretation of the learned policies, quantifies the trade-offs between computational overhead and solution quality, and outlines current limitations of the model along with directions for future work.

A. Learned State-Conditioned Neighborhood Selection

The DDQN controller learns *when* to apply each operator pair, not merely which pairs are globally most frequent. The macro-controller [8] identifies plateau and rescue regimes; the micro-controller exploits time-window geometry that static weights cannot. This is why Hybrid-DDQN matches Hybrid-Rule exactly on vehicle count (paired Wilcoxon $p = 1.0$; identical on all 56 Solomon instances) while improving on distance at matched fleet size (+1.134% vs. +1.350% over the $N=50$ instances both solve at $NV \leq NV_{BKS}$, one-sided Wilcoxon $p = 0.064$)—a consistent but, at this sample size, only marginally significant distance edge.

Zero-shot transfer is decisive evidence of generalisation: weights are trained on synthetic domain-randomized graphs [10] and never tuned on Solomon or Gehring–Homberger data.

B. Computational Cost and Suboptimal Scaling Behavior

At the 400-customer scale and beyond, neither heuristic reaches the best-known solutions in the literature—on `r2_4_1` ($NV_{BKS} = 4$) both solvers land near 8 vehicles—reflecting a genuine scalability ceiling for constructive and local-search heuristics. The contribution at these scales is therefore relative, not absolute: Hybrid-DDQN degrades more gracefully than ALNS-Base, with the fleet-count edge widening from 0.46 vehicles at $N=400$ to 1.78 at $N=1000$ (§IV), and the effect is significant on the larger 24-instance shards ($p \leq 3.9 \times 10^{-3}$) even though the six-instance shards at $N=800, 1000$ individually do not reach significance.

The route-minimization machinery buys these vehicle savings at a real cost in wall-clock time: the Guided Ejection Search and wide-time-window polish make Hybrid-DDQN’s tail markedly more expensive than ALNS-Base’s, so on instances where ALNS-Base early-stops the two are no longer iso-effort. This is why the scalability shards are reported under a fixed anytime budget: it holds wall-clock time equal across solvers, which is the fair basis for the OR-Tools comparison above and the honest way to price the macro-policy’s extra search against its fleet reduction.

C. Limitations

RC101 Pareto alternative: Convergence to $NV = 15$ (vs. $NV_{BKS} = 14$) with total distance 2.6% below the literature reference [5] is a Pareto trade-off, not a failure.

Wide time-window gradient sparsity: R2 and RC2 families produce sparse reward surfaces because many service orderings share similar marginal costs. This explains the absent NV separation between Hybrid-DDQN and Hybrid-Fixed (identical vehicle counts), even though distance improvement remains clear (+3.223% vs. +4.677% on the wide-horizon fair subset).

Compute–quality trade-off: Hybrid-DDQN is 2–100× slower than ALNS-Base on instances where early stopping would otherwise terminate search. This overhead is the cost of the macro-policy’s route-reduction exploration and should be weighed against the fleet-size savings.

Large-scale overhead: The $14\times$ Numba-JIT speedup makes the 200-customer pilot practical, but graph-encoding optimizations [13] are required for 1000-customer deployment.

VI. CONCLUSION

Hybrid DDQN-ALNS replaces static roulette-wheel selection with a hierarchical dual-agent controller that learns state-conditioned operator policies across a domain-randomized curriculum. The architecture enforces strict lexicographic fleet-size discipline while achieving a 13.3% reduction in vehicle inflation over ALNS-Base, a 92.7% reduction over OR-Tools [15], and a 5.8% reduction in wide-horizon travel distance gap—all verified under zero-shot transfer to unseen benchmarks.

Future work will validate the remaining * components (N-step returns, HER relabelling, adaptive curriculum) via ablation on the Solomon and Gehring–Homberger suites, replace handcrafted state features with attention-based graph encodings [12], [13], and extend the model to dynamic, real-time routing fleets.

ACKNOWLEDGMENT

The authors thank Dr. Ho Thi Linh for her guidance throughout this work.

REFERENCES

- [1] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [2] S. Ropke and D. Pisinger, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.
- [3] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” in *Principles and Practice of Constraint Programming (CP 1998)*, LNCS 1520, pp. 417–431, Springer, 1998.
- [4] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
- [5] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
- [6] H. Gehring and J. Homberger, “A parallel two-phase metaheuristic for routing problems with time windows,” in *Proc. Asia-Pacific Decision Sciences Institute Conf.*, pp. 114–124, 1999.
- [7] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [8] Z. Wang *et al.*, “Dueling network architectures for deep reinforcement learning,” in *Proc. 33rd ICML*, vol. 48, pp. 1995–2003, 2016.
- [9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Int. Conf. Learning Representations (ICLR)*, 2016.
- [10] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ IROS*, pp. 23–30, 2017.
- [11] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
- [12] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in NeurIPS*, vol. 28, pp. 2692–2700, 2015.
- [13] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *Int. Conf. Learning Representations (ICLR)*, 2019.
- [14] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, “Reinforcement learning for solving the vehicle routing problem,” in *Advances in NeurIPS*, pp. 9861–9871, 2018.
- [15] Google LLC, “OR-Tools: Open source software for combinatorial optimization,” 2023. [Online]. Available: <https://developers.google.com/optimization>

- [16] Y. Nagata and O. Bräysy, "A powerful route minimization heuristic for the vehicle routing problem with time windows," *Operations Research Letters*, vol. 37, no. 5, pp. 333–338, 2009.
- [17] Y. Nagata, O. Bräysy, and W. Dullaert, "A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows," *Computers & Operations Research*, vol. 37, no. 4, pp. 724–737, 2010.